



Instituto Politécnico do Cávado e do Ave

Escola Superior De Tecnologia

Licenciatura em Engenharia de Sistemas Informáticos

UC: Estrutura de Dados Avançadas

Relatório de Trabalho Prático – Fase 1 & 2

Aluno: Rúben Manuel Abreu de Oliveira, a24861

Docente Responsável: Luis Gonzaga Martins Ferreira

Ano Letivo: 2024/2025

Repositório GitHub: <https://github.com/RubenO94/EDA-TP/tree/v-1-0>

Março, 2025

Resumo

O presente relatório descreve o desenvolvimento de uma aplicação em linguagem C que simula a representação e análise de antenas numa cidade, através de uma estrutura de dados dinâmica baseada em listas ligadas. O projeto foi desenvolvido no âmbito da unidade curricular de Estruturas de Dados Avançadas e dividido em duas fases principais.

Na Fase 1, o foco centrou-se na construção e manipulação de listas ligadas simples, com operações de criação, inserção, pesquisa, remoção e libertação. A lógica de negócio associada à representação de antenas foi proposta livremente pelos alunos, permitindo aplicar os fundamentos lecionados de forma criativa e contextualizada.

A Fase 2 introduziu uma estrutura de grafo, onde cada antena é representada como um vértice com coordenadas e frequência. Foram implementadas funcionalidades como: carregamento de uma matriz de texto para criar o grafo, ligação automática entre vértices adjacentes com a mesma frequência, percursos BFS e DFS, e persistência do grafo através de ficheiros binários com controlo de integridade.

A funcionalidade de análise de interseções entre frequências distintas foi deixada de fora desta fase, uma vez que apenas foi referida de forma teórica em aula, sem aprofundamento ou prática. A decisão de não as incluir foi feita para garantir domínio total do código entregue.

Este trabalho permitiu consolidar competências práticas na implementação e análise de estruturas dinâmicas, promover boas práticas de organização modular em C, e aplicar percursos e operações em grafos dentro de um contexto aplicacional.

Tabela de Conteúdo

Capítulo 1 – Introdução	1
Motivação	2
Enquadramento	2
Objetivos	3
Metodologias de Trabalho	4
Plano de Trabalho	5
Estrutura do Documento	6
Capítulo 2 - Revisão da Literatura e Enquadramento Teórico	8
Estruturas de Dados Dinâmicas	8
Gestão de Memória em Linguagem C	9
Documentação com Doxygen	10
Ferramentas Utilizadas	11
Capítulo 3 – Análise e Especificação	12
Requisitos	12
Requisitos Funcionais	12
Requisitos Não Funcionais	13
Regras de Negócio e Validações	13
R1 – Posição Única por Antena	13
R2 – Frequência Case-Insensitive	14
R3 – Identificação de Interferência	14
R4 – Representação de Efeitos Nefastos	14
Arquitetura Lógica	15

Organização Modular do Código.....	16
Fluxo de Execução da Aplicação.....	17
Interação e Fluxo de Operações.....	18
Capítulo 4 – Implementação	19
Estrutura Geral do Projeto	19
Principais Funcionalidades Implementadas.....	20
Boas Práticas e Organização (<i>reforçada</i>)	21
Capítulo 4 – Fase 2: Estrutura de Grafos e Algoritmos.....	21
Reorganização do Projeto	22
Justificação da Nova Estrutura	22
Funcionalidades Implementadas na Fase 2.....	23
Decisões Técnicas Relevantes	23
Capítulo 5 – Análise e Discussão de Resultados	24
Decisões Técnicas Justificadas	24
Dificuldades Técnicas Sentidas	24
Falhas Identificadas e Aprendizagens	25
Lições Aprendidas e Melhoria Contínua	26
Propostas de Melhoria Futuras	26
Utilização de apontador tail	26
Reestruturação de funções com lógica repetida.....	26
Generalização com apontadores para função.....	27
Considerações Finais da Fase 2	27
Capítulo 6 – Conclusão.....	28
Referências	29

Capítulo 1 – Introdução

O presente capítulo pretende apresentar o contexto e os objetivos do trabalho desenvolvido no âmbito da unidade curricular de Estruturas de Dados Avançadas. Para tal, começa-se por explicar a motivação para a realização deste projeto, identificando a sua relevância prática e académica. Em seguida, é descrito o enquadramento do trabalho no percurso académico do aluno, os objetivos propostos, a metodologia adotada para o seu desenvolvimento e o plano de trabalho. Por fim, é apresentada a estrutura do relatório.

Motivação

A motivação para o desenvolvimento deste trabalho está diretamente relacionada com a consolidação dos conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas, em especial no que diz respeito à implementação e manipulação de estruturas dinâmicas em linguagem C.

Na primeira fase, o foco centrou-se nas listas ligadas, uma das estruturas fundamentais da informática, permitindo explorar técnicas de gestão eficiente de memória e dados em contextos dinâmicos, com ênfase na alocação/libertação de memória e no domínio de apontadores. Através da construção de uma lógica aplicada ao contexto das antenas, foi possível integrar os conceitos técnicos a uma modelação orientada ao problema.

Na segunda fase, a complexidade evoluiu naturalmente para grafos, exigindo uma abstração superior e novas estruturas como listas de adjacência. A aplicação de algoritmos de percurso (BFS e DFS) trouxeram novos desafios relacionados com o desempenho da aplicação. Esta progressão permitiu aplicar os fundamentos de estruturas dinâmicas num cenário mais próximo da resolução de problemas reais.

O trabalho, no seu conjunto, visa não apenas consolidar os conhecimentos técnicos da unidade curricular, mas também desenvolver autonomia, capacidade de análise e boas práticas de organização e documentação de código.

Enquadramento

Este trabalho foi desenvolvido no âmbito da unidade curricular de Estruturas de Dados Avançadas, pertencente ao primeiro ano da Licenciatura em Engenharia de Sistemas Informáticos, lecionada no Instituto Politécnico do Cávado e do Ave (IPCA).

O projeto aqui documentado corresponde à totalidade das fases propostas no trabalho prático da unidade curricular. A Fase 1 teve como objetivo a implementação de listas ligadas dinâmicas, com operações básicas de criação, inserção, pesquisa e remoção, aplicadas ao contexto de uma rede de antenas. Já a Fase 2 introduziu o

conceito de grafo, expandindo a estrutura inicial para um sistema mais complexo, com representação de vértices (antenas), ligações condicionadas por frequência, e algoritmos de percurso como busca em largura (BFS) e profundidade (DFS).

Esta evolução natural permitiu aplicar, de forma progressiva, os conteúdos lecionados sobre estruturas de dados dinâmicas, reforçando os conhecimentos teóricos e desenvolvendo competências práticas essenciais no contexto da programação em linguagem C e da gestão de memória dinâmica. O trabalho culmina numa aplicação modular, funcional e alinhada com as exigências da disciplina.

Objetivos

Os principais objetivos definidos para o desenvolvimento deste trabalho prático foram os seguintes:

- Consolidar os algoritmos fundamentais estudados em aula, nomeadamente:
 - criação de nós dinamicamente
 - inserção ordenada
 - pesquisa e remoção
 - libertar memória em listas ligadas
- Implementar uma estrutura dinâmica que represente uma rede de antenas posicionadas num plano cartesiano, com coordenadas (x, y) e uma frequência associada a cada antena.
- Evoluir a estrutura inicial para um grafo dinâmico, onde cada vértice corresponde a uma antena e cada aresta representa uma ligação válida entre antenas com a mesma frequência e em posições adjacentes.
- Aplicar algoritmos de percurso em grafos (BFS e DFS), com respeito às restrições impostas pelas frequências das antenas, analisando os vértices alcançáveis a partir de um ponto inicial.
- Permitir a persistência do estado do grafo através de ficheiros de texto (.txt) e ficheiros binários (.bin), garantindo a integridade dos dados lidos e escritos.

- Manter a modularidade, organização e legibilidade do código, respeitando boas práticas de programação em linguagem C.
- Entregar uma solução coerente com o conteúdo lecionado em aula, priorizando a compreensão e explicação do código implementado.

Metodologias de Trabalho

A metodologia de trabalho adotada ao longo das duas fases deste projeto assentou numa abordagem iterativa e incremental, permitindo um desenvolvimento progressivo e controlado das estruturas e funcionalidades pretendidas.

Na **Fase 1**, o foco centrou-se na implementação de operações base sobre listas ligadas, conforme ensinadas em aula, tais como: criação de nós, inserção ordenada, pesquisa por coordenadas, remoção de elementos e libertação dinâmica de memória. Com base nessas operações, foi criada uma lógica de negócio que simulava a deteção de interferência entre antenas, aplicando regras definidas pelo aluno. A estrutura foi organizada de forma modular, separando as responsabilidades por ficheiros e funções, garantindo clareza e escalabilidade.

Na **Fase 2**, essa base foi expandida com a introdução de grafos, exigindo uma nova modelação orientada a vértices e arestas. A implementação de listas de adjacência permitiu representar relações de proximidade entre antenas com a mesma frequência. Foram aplicados algoritmos de percurso (BFS e DFS), e o grafo passou a suportar persistência de estado através de ficheiros binários e de texto.

Durante todo o processo, foi dada especial atenção à modularidade do código, à validação de dados em tempo de execução e à manutenção da integridade da memória. Foram realizados testes manuais utilizando matrizes de antenas complexas, desenhadas para validar casos de ligação vertical, horizontal e diagonal. Sempre que aplicável, foi usada a ferramenta Doxygen para documentar o código-fonte de forma estruturada e manter a documentação técnica alinhada com a implementação.

Plano de Trabalho

O enunciado da Fase 1 do trabalho prático foi disponibilizado no dia **14 de março de 2025**, com data de entrega estipulada para **30 de março de 2025**. Dado o tempo reduzido para o desenvolvimento completo e documentação detalhada, o foco inicial recaiu sobre a implementação funcional da estrutura de dados e da lógica de negócio, com testes manuais para validar o comportamento do sistema.

Na data prevista, foi entregue um relatório preliminar em formato *Markdown*, que cobria os aspetos essenciais do trabalho, mas com uma descrição sucinta e sem o rigor técnico e formal exigido num relatório académico completo. Por esse motivo, e com o objetivo de melhorar a qualidade da documentação, decidiu-se posteriormente reestruturar e reescrever o relatório, seguindo as normas de redação técnica e científica recomendadas.

Com o início da **Fase 2, em abril de 2025**, o trabalho evoluiu naturalmente para estruturas de grafos, exigindo novas abordagens de modelação, algoritmos de percurso e persistência em ficheiros binários. Esta fase manteve a metodologia iterativa e incremental, garantindo que cada funcionalidade era compreendida, testada e documentada antes de avançar para a seguinte.

A distribuição do tempo ficou organizada da seguinte forma:

- **14 a 29 de março (Fase 1):**
 - Implementação das operações sobre listas ligadas
 - Desenvolvimento da lógica de negócio (frequência, interferências, efeitos)
 - Documentação técnica inicial com Doxygen
- **29 a 30 de março:**
 - Testes manuais e entrega de relatório simplificado (formato Markdown)
- **Abril a maio de 2025 (Fase 2):**
 - Expansão da estrutura para grafos (vértices e adjacências)
 - Implementação de ``graph_connect_adjacent_antennas``, BFS e DFS
 - Escrita e leitura de ficheiros ``*.txt`` e ``*.bin`` com validação de integridade

- Testes com matrizes complexas e validação de percursos
- Organização modular e revisão contínua com Doxygen
- **Maio de 2025:**
 - Reescrita completa do relatório, com secções académicas e justificação de decisões
 - Consolidação da documentação técnica e eliminação de funcionalidades não dominadas

Estrutura do Documento

Este relatório está organizado em seis capítulos, para além da presente introdução. A estrutura adotada visa apresentar de forma clara e progressiva todo o trabalho desenvolvido, desde a fundamentação teórica até à análise dos resultados obtidos.

No **Capítulo 2** é realizada uma revisão dos principais conceitos teóricos e técnicos relevantes para a implementação do projeto, incluindo estruturas de dados dinâmicas, listas ligadas e manipulação de memória em linguagem C.

O **Capítulo 3** apresenta a análise do problema e a especificação da solução, incluindo os requisitos definidos, a lógica de negócio associada às regras de interferência entre antenas, e a modelação da estrutura de dados desenvolvida.

No **Capítulo 4**, é detalhada a implementação da aplicação, descrevendo-se a organização dos módulos, as principais funções desenvolvidas e as decisões técnicas tomadas durante o processo de desenvolvimento.

O **Capítulo 5** foca-se na análise e discussão dos resultados obtidos, apresentando casos de teste relevantes, validando o comportamento do sistema e refletindo sobre as suas limitações e possíveis melhorias.

Por fim, o **Capítulo 6** apresenta as conclusões do trabalho, destacando os objetivos atingidos, as dificuldades encontradas e as propostas para trabalhos futuros.

O relatório inclui ainda uma secção de **Referências Bibliográficas**, com todas as fontes consultadas, bem como anexos com documentação de apoio e exemplos práticos da estrutura desenvolvida

Capítulo 2 - Revisão da Literatura e Enquadramento Teórico

O presente capítulo tem como objetivo apresentar os principais conceitos teóricos e técnicos que sustentam o trabalho desenvolvido. Para isso, são abordadas estruturas de dados dinâmicas com foco nas listas ligadas, bem como técnicas de gestão de memória em linguagem C. Adicionalmente, são exploradas as ferramentas utilizadas para apoiar o desenvolvimento e documentação do projeto, e, por fim, são referidas algumas abordagens semelhantes ou comparáveis, com uma breve análise comparativa.

Estruturas de Dados Dinâmicas

As estruturas de dados dinâmicas desempenham um papel essencial no desenvolvimento de aplicações que exigem flexibilidade no armazenamento e manipulação de dados. Ao contrário das estruturas estáticas, como os arrays, as estruturas dinâmicas permitem que a memória seja alocada e libertada durante a execução do programa, ajustando-se às necessidades reais do sistema em tempo de execução(Weiss, 2011).

Entre as estruturas dinâmicas mais utilizadas, as **listas ligadas** (ou linked lists) destacam-se pela sua simplicidade e eficiência em operações como inserção e remoção de elementos. Numa lista ligada, cada nó contém os dados relevantes e um apontador para o próximo nó da sequência, o que elimina a necessidade de mover elementos ao inserir ou remover (Horowitz et al., 2009). Esta característica torna as listas ideais para cenários onde a dimensão dos dados não é conhecida previamente ou pode variar com frequência.

A utilização de listas ligadas é especialmente recomendada quando se pretende manter um conjunto de elementos ordenados com base em critérios dinâmicos, como ocorre no projeto aqui descrito, onde as antenas são organizadas por coordenadas. A abordagem adotada baseia-se numa lista simplesmente ligada, suficiente para percorrer sequencialmente os elementos, verificar interferências entre antenas e representar efeitos nefastos. Esta escolha foi influenciada pela clareza de implementação e pela eficiência em memória, aspetos amplamente discutidos na literatura (Knuth, 1997).

Gestão de Memória em Linguagem C

A linguagem C oferece ao programador um controlo total sobre a alocação e libertação de memória, o que constitui simultaneamente uma vantagem e uma responsabilidade. Ao contrário de linguagens modernas com gestão automática de memória (garbage collection), em C é necessário recorrer explicitamente a funções como *malloc* e *free*, todas incluídas na biblioteca *<malloc.h>* (Damas, 2002).

A correta gestão de memória é crucial para garantir a estabilidade, desempenho e fiabilidade de aplicações em C. Erros nesta área podem resultar em **memory leaks**, **acessos inválidos a memória libertada** (dangling pointers) ou ainda **corrupção de memória**, problemas que, embora comuns, são muitas vezes difíceis de detetar durante a execução normal do programa (Weiss, 2011).

No âmbito do presente projeto, a gestão de memória foi uma das componentes centrais, uma vez que a estrutura principal — uma lista simplesmente ligada — exige que cada nó seja alocado dinamicamente no momento da inserção, e libertado corretamente aquando da sua remoção ou destruição da lista. Para tal, recorreu-se à separação clara entre as funções de criação de nós e as de inserção, promovendo o encapsulamento das responsabilidades e reduzindo o risco de falhas associadas à manipulação de apontadores.

A necessidade de libertar todos os nós antes de terminar a execução do programa levou à implementação de uma função de destruição completa da lista, que percorre todos os elementos e liberta a memória associada a cada um. Esta prática segue as recomendações de boas práticas em programação em C, que defendem que qualquer memória alocada deve, sem exceção, ser libertada antes do encerramento do programa (Horowitz et al., 2009).

Documentação com Doxygen

Uma das boas práticas na engenharia de software é a manutenção de documentação técnica clara e atualizada, especialmente em projetos desenvolvidos em linguagens como C, onde a legibilidade e a organização do código nem sempre são garantidas pela própria linguagem. Para este fim, foi utilizada a ferramenta **Doxygen**, uma aplicação amplamente adotada para a **criação automática de documentação técnica** a partir de comentários inseridos diretamente no código-fonte.

O Doxygen permite gerar documentação em diversos formatos (HTML, PDF, RTF, LaTeX, entre outros) e suporta linguagens como C, C++, Java, Python, entre outras. No contexto do presente projeto, foi utilizado para produzir documentação estruturada do código desenvolvido em C, incluindo listas de funções, estruturas de dados, diagramas de dependência, e explicações associadas a cada componente do sistema.

A utilização do Doxygen ao longo do desenvolvimento revelou-se fundamental para manter uma visão clara da estrutura do código, facilitando a sua organização e manutenção. Para tal, foram utilizados blocos de comentários com marcações especiais (como `/** */`, `@param`, `@return`, etc.), permitindo que o Doxygen extraísse automaticamente a informação necessária para compor a documentação.

Esta abordagem não só melhorou a legibilidade e navegabilidade do código, como também promoveu o rigor na escrita de funções bem definidas e devidamente comentadas, seguindo os princípios recomendados por autores como (Martin, 2008),

que defende que “*código limpo é aquele que foi bem escrito e bem pensado*”, e que a documentação deve complementar — e não substituir — a clareza do código em si.

Ferramentas Utilizadas

O desenvolvimento do projeto foi suportado por um conjunto de ferramentas que permitiram implementar, testar e documentar a solução de forma eficiente e estruturada.

Para a edição e organização do código, foi utilizado o editor **Visual Studio Code**, com suporte para extensões de realce de sintaxe, e integração com o terminal. A compilação e execução do programa foram feitas num ambiente Linux, através da utilização do compilador **GCC (GNU Compiler Collection)**. Esta escolha deveu-se à sua fiabilidade, ao suporte de padrões modernos da linguagem C e à facilidade de automação de tarefas de compilação através de **Makefiles**.

O controlo de versões foi realizado localmente com **Git**, de forma a manter o histórico de alterações e possibilitar a reversão de estados anteriores do código em caso de erro. A utilização do Git revelou-se útil, mesmo sendo um projeto individual, na organização das fases de implementação.

A documentação técnica do código foi produzida com **Doxygen**, como referido na secção anterior, permitindo criar documentação técnica e manter o código comentado com clareza. Para a redação do relatório final, foi utilizada a aplicação **Microsoft Word**, com formatação de acordo com as normas académicas indicadas pela unidade curricular.

Capítulo 3 – Análise e Especificação

O presente capítulo tem como objetivo apresentar a análise do problema abordado neste projeto, bem como a especificação técnica da solução proposta. São identificados os requisitos funcionais e as principais regras de negócio que regem o comportamento da aplicação, e é feita uma descrição da arquitetura lógica adotada. A modelação das estruturas de dados e a forma como estas interagem entre si são também abordadas nesta secção, preparando o leitor para a compreensão detalhada da implementação apresentada no capítulo seguinte.

Requisitos

Requisitos Funcionais

Os requisitos funcionais definem as funcionalidades mínimas que o sistema deve implementar para cumprir os objetivos definidos:

- **RF1** – Permitir a criação de uma estrutura dinâmica do tipo lista ligada para armazenar os elementos.
- **RF2** – Inserir novos elementos (antenas) na lista, com coordenadas x e y e uma frequência associada.
- **RF3** – Verificar se a posição de inserção já está ocupada por outra antena.
- **RF4** – Identificar automaticamente situações de interferência entre antenas da mesma frequência, se a distância entre elas for inferior ao valor definido como mínimo aceitável.
- **RF5** – Registar os efeitos nefastos da interferência, através da criação de novos elementos (com a frequência #) nas posições afetadas.
- **RF6** – Permitir remover elementos da lista, incluindo antenas e efeitos nefastos.
- **RF7** – Permitir a pesquisa de elementos com base nas coordenadas.
- **RF8** – Libertar corretamente toda a memória alocada, destruindo a lista no fim da execução.

- **RF9** – Organizar o código de forma modular, com separação clara entre criação de nós, inserção, validação, remoção e impressão da estrutura.

Requisitos Não Funcionais

Além das funcionalidades, foram também definidos alguns critérios de qualidade a observar:

- **RNF1** – O código deve ser escrito em linguagem C.
- **RNF2** – O código deve seguir boas práticas de legibilidade, modularidade e gestão de memória.
- **RNF3** – A documentação técnica do código deve ser gerada automaticamente com Doxygen.
- **RNF4** – O sistema deve permitir fácil manutenção e expansão para fases futuras.

Regras de Negócio e Validações

O funcionamento da estrutura desenvolvida baseia-se num conjunto de regras de negócio que garantem a integridade dos dados e a coerência do sistema. Estas regras foram definidas com base no enunciado da fase 1 do projeto, complementadas por decisões tomadas pelo aluno durante a análise e implementação.

R1 – Posição Única por Antena

Não é permitida a existência de duas antenas na mesma posição. Assim, ao tentar inserir uma antena numa coordenada (x, y) já ocupada por outra antena, a inserção é rejeitada. Esta validação evita duplicações e garante que cada antena representa uma localização única no plano.

R2 – Frequência Case-Insensitive

A frequência de cada antena é representada por um caractere do alfabeto. No entanto, para efeitos de comparação, a frequência é considerada **insensível a maiúsculas e minúsculas** (ex: **A** e **a** são tratadas como iguais). Esta decisão foi adotada para simplificar a lógica de interferência e evitar duplicações implícitas.

R3 – Identificação de Interferência

Sempre que duas ou mais antenas com a **mesma frequência** são inseridas e a **distância entre elas for inferior a N unidades**, considera-se que há interferência. A distância é calculada com base na **distância Euclidiana** entre os pontos:

$$\text{distância} = \sqrt{\{(x_2 - x_1)^2 + (y_2 - y_1)^2\}}$$

Se a distância for inferior ao valor mínimo aceitável (N), o sistema deteta automaticamente a interferência e prossegue com a criação de efeitos nefastos.

R4 – Representação de Efeitos Nefastos

A interferência detetada dá origem à criação de elementos do tipo **efeito nefasto**, representados com o caractere **#** como frequência. Os efeitos nefastos são inseridos automaticamente em todas as posições **entre as antenas em conflito**, respeitando a simetria e propagação do efeito.

- Se a interferência for a 2 unidades de distância, o efeito propaga-se 4 posições entre as duas antenas.
- A regra geral adotada foi: **interferência** $\rightarrow 2 \times$ (**distância entre antenas**) **posições afetadas**

Os efeitos nefastos podem ser criados mesmo em posições já ocupadas por outras antenas ou por efeitos anteriores, sendo o símbolo # considerado prioritário para marcar zonas de conflito.

Arquitetura Lógica

A aplicação foi desenvolvida com uma arquitetura modular, baseada na separação de responsabilidades por ficheiros e funções específicas. Esta abordagem permite melhorar a legibilidade, facilitar a manutenção do código e preparar a base para possíveis evoluções do sistema em fases futuras.

O núcleo da aplicação assenta em duas listas ligadas distintas:

- Uma lista para armazenar **antenas** (com coordenadas e frequência)
- Uma lista separada para armazenar **efeitos nefastos** causados por interferência

Ambas as listas utilizam a **mesma estrutura de dados (struct)**, que contém os campos:

```
typedef struct ED
{
    char frequency;
    int x, y;
    struct ED *next;
} ED;
```

Ilustração 1 - Struct ED

Esta decisão permite **reutilizar as funções de criação, pesquisa e remoção**, mantendo uma lógica comum para o tratamento de elementos no plano, enquanto se distingue claramente entre tipos de elementos através do conteúdo da frequência ou pela lista onde estão inseridos.

Organização Modular do Código

A seguir apresenta-se um resumo dos módulos existentes e das suas responsabilidades:

- **ed.c / ed.h**

Contém as funções principais de manipulação da lista ligada (criação, inserção, remoção, pesquisa, destruição), comuns tanto a antenas como a efeitos nefastos, visto que ambas as listas usam a mesma struct.

- **ed_validator.c / ed_validator.h**

Responsável pelas validações de posições e frequências, incluindo detecção de duplicações, limites do plano e consistência dos dados inseridos.

- **geometry.c / geometry.h**

Implementa funções matemáticas auxiliares, como o cálculo da **distância Euclidiana**, central para a lógica de interferência.

- **effect.c / effect.h**

Contém a lógica para criação e inserção de efeitos nefastos (frequency = '#'), com base na distância entre antenas da mesma frequência.

- **controller.c / controller.h**

Responsável pela coordenação geral das operações entre módulos (ex: inserção de antenas, verificação de regras e gestão das listas).

- **display.c / display.h**

Contém as funções de visualização da estrutura no terminal, com suporte a **cores** através do módulo terminal_colors.

- **menu.c / menu.h**

Define a interface textual com o utilizador, com base em menus e opções interativas.

- **ed_io.c / ed_io.h**

Prevê a leitura e escrita de ficheiros para persistência de dados (a ser explorado em fases futuras).

- **constants.h**

Centraliza constantes globais utilizadas pelos vários módulos, como limites de distância, dimensões do plano, etc.

- **terminal_colors.h**

Define macros e strings ANSI para a aplicação de cores no terminal, melhorando a visualização de elementos.

Fluxo de Execução da Aplicação

O ponto de entrada da aplicação é o ficheiro **main.c**, que contém a função *main()*. Nesta função, é invocada a função *start_menu()*, definida em *menu.c*, responsável por apresentar o menu principal e gerir a interação com o utilizador.

O módulo *menu.c* está encarregue da navegação pelas diferentes opções da aplicação. Para executar as funcionalidades escolhidas pelo utilizador (como inserir uma antenna, listar elementos, remover posições, etc.), este módulo invoca diretamente funções definidas no módulo *controller.c*.

O ficheiro *controller.c* atua como **orquestrador central** do sistema. Através das funções declaradas em *controller.h*, este módulo articula a comunicação entre os diversos componentes especializados: manipulação de listas (*ed.c*), validações (*ed_validator.c*), lógica geométrica (*geometry.c*), criação de efeitos nefastos (*effect.c*) e visualização (*display.c*).

Esta organização promove uma arquitetura em camadas, onde a interação com o utilizador é separada da lógica interna da aplicação, e onde cada módulo tem uma responsabilidade bem definida. Tal abordagem facilita a manutenção do sistema e o isolamento de erros, alinhando-se com boas práticas de desenvolvimento modular (Martin, 2008).

Interação e Fluxo de Operações

A aplicação foi desenvolvida com base numa interface textual simples, permitindo ao utilizador inserir antenas, visualizar a estrutura, remover elementos e sair do programa. Ao inserir uma antena, o sistema valida a posição, compara a frequência com as existentes e, se detetada interferência, gera automaticamente os efeitos nefastos. Todas as operações são coordenadas por um menu principal e conduzidas por um controlador central, que comunica com os módulos responsáveis por cada funcionalidade.

Capítulo 4 – Implementação

O presente capítulo descreve a implementação do sistema desenvolvido, com base na análise e especificações apresentadas anteriormente. São abordadas as principais decisões técnicas, a estrutura geral do código, as funcionalidades implementadas, bem como algumas boas práticas aplicadas ao longo do desenvolvimento. Pretende-se com isto dar ao leitor uma visão clara sobre a forma como o projeto foi construído em termos práticos.

Estrutura Geral do Projeto

A aplicação foi desenvolvida em linguagem C, com base numa arquitetura modular. O código encontra-se dividido em múltiplos ficheiros .c e .h, organizados nos diretórios:

- **src/** – implementação das funcionalidades
- **include/** – cabeçalhos com definições e assinaturas
- **docs/** – relatório e documentação gerada automaticamente pelo Doxygen
- **input/** – ficheiros auxiliares de entrada

Cada módulo lida com uma responsabilidade específica. Por exemplo, o módulo `ed.c` trata da gestão de listas ligadas, enquanto o `geometry.c` trata do cálculo de distâncias. Esta organização modular permitiu manter o código limpo, reutilizável e facilmente navegável.

Decidiu-se manter a estrutura do projeto separada por diretórios, pois esta abordagem facilita a manutenção, promove a organização e permite localizar rapidamente funções específicas, algo que se tornaria essencial em fases futuras mais complexas.

Principais Funcionalidades Implementadas

Entre as funcionalidades desenvolvidas destacam-se:

- Criação de elementos dinâmicos, com alocação de memória controlada
- Inserção de antenas, com verificação de posições já ocupadas
- Cálculo da distância Euclidiana entre antenas para detecção de interferência
- Criação automática de efeitos nefastos (#) em zonas de conflito
- Remoção de elementos (antenas e efeitos) por coordenadas
- Libertação de memória no encerramento da aplicação
- Interface textual baseada em menus
- Visualização gráfica no terminal, com uso de cores

A inserção dos elementos na lista foi implementada com um algoritmo que mantém a lista ordenada com base nas coordenadas, de forma a facilitar a leitura e garantir um comportamento previsível. Esta decisão visou também preparar o projeto para futuras funcionalidades que beneficiem da ordenação.

Optou-se por isolar a lógica de criação de nós da lógica de inserção, mantendo as funções com responsabilidades únicas, conforme as boas práticas de desenvolvimento modular. Cada função foi projetada para executar apenas uma tarefa clara, como validar, criar, inserir ou verificar conflitos.

Boas Práticas e Organização *(reforçada)*

Durante o desenvolvimento foram seguidas boas práticas:

- Funções pequenas e coesas, cada uma com uma única responsabilidade
- Separação clara entre criação, validação, inserção e visualização
- Modularização do código por ficheiros
- Nomes de variáveis e funções em inglês, legíveis e consistentes
- Uso de constantes globais centralizadas
- Comentários estruturados compatíveis com Doxygen

Evitaram-se funções com múltiplas responsabilidades, seguindo o princípio da responsabilidade única (SRP). Por exemplo, a função de inserção apenas insere o nó ordenadamente; a validação da posição foi feita noutra função antes da chamada. Esta abordagem contribuiu para um código mais fácil de testar e manter.

Capítulo 4 – Fase 2: Estrutura de Grafos e Algoritmos

Este capítulo documenta o desenvolvimento da Fase 2 do projeto, onde a estrutura inicial baseada em listas ligadas foi expandida para suportar grafos com representação de antenas, ligação entre vértices, e percursos como BFS e DFS. Embora o código se mantenha no mesmo projeto, a complexidade dos requisitos exigiu alterações profundas na arquitetura do sistema, incluindo a separação em submódulos e diretorias, assim como a criação de novas estruturas mais adequadas ao modelo de grafo.

Reorganização do Projeto

Com o aumento da complexidade na Fase 2, o projeto foi reorganizado para garantir maior modularidade e escalabilidade. A pasta ``src/`` passou a conter uma subpasta ``lib/`` com os ficheiros fonte (.c), enquanto o diretório ``include/`` foi criado para conter todos os headers públicos do projeto (`graph.h`, `algorithms.h`, `queue.h`, `graph_io.h`, etc.).

Foi ainda gerada uma biblioteca estática ``libed.a`` a partir do código compilado, permitindo separar o ponto de entrada (`main.c`) da lógica do sistema, seguindo boas práticas de organização por camadas.

Justificação da Nova Estrutura

Apesar de existir uma estrutura ED já implementada na Fase 1 (``struct ED`` em `ed.h`), optou-se por não a reaproveitar diretamente como vértice do grafo, uma vez que esta estrutura foi concebida com lógica de lista ligada simples (com um ``next`` explícito), o que poderia introduzir ambiguidades ao representá-la também como nó de grafo com lista de adjacência.

Assim, foi criada uma estrutura ``Vertex``, mais clara e dedicada ao modelo de grafo, contendo frequência, coordenadas, ``visited`` e apontadores para adjacências (``Adj_Node *adj_list``) e para o próximo vértice (``Vertex *next``), mantendo a coesão interna do grafo.

```
typedef struct Vertex
{
    char frequency;
    int x, y, visited;
    struct Adj_Node *adj_list;
    struct Vertex *next;
};
```

Ilustração 2 - Struct Vertex

Funcionalidades Implementadas na Fase 2

- Leitura de uma matriz .txt e construção automática de vértices
- Criação de adjacências entre vértices da mesma frequência em posições adjacentes
- Implementação de algoritmos BFS e DFS restritos à mesma frequência
- Escrita e leitura binária do grafo, com marcador de fim de vértice
- Impressão dos vértices visitados e do grafo carregado

Decisões Técnicas Relevantes

- Não reutilizar ``struct ED`` para evitar ambiguidade entre conceitos distintos
- Modularizar as funcionalidades em ficheiros dedicados: ``graph.c``, ``graph_io.c``, ``algorithms.c``, etc.
- Colocar headers em ``include/`` para facilitar documentação com Doxygen
- Priorizar clareza e explicabilidade em detrimento da reutilização forçada

Capítulo 5 – Análise e Discussão de Resultados

Este capítulo apresenta uma reflexão crítica sobre o trabalho desenvolvido, tendo por base as decisões técnicas tomadas, os desafios encontrados ao longo da implementação, as falhas identificadas e as oportunidades de melhoria detetadas. A análise tem como objetivo evidenciar a capacidade de identificar fragilidades no código, justificar escolhas técnicas e reconhecer o processo de aprendizagem envolvido.

Decisões Técnicas Justificadas

Durante a implementação, foram tomadas decisões com base na clareza, eficiência e modularidade do código. Entre elas destacam-se:

- **Separação entre listas de antenas e efeitos nefastos**, permitindo tratamento independente e facilitando a visualização e limpeza das estruturas.
- **Inserção ordenada por coordenadas**, adotada para melhorar a organização visual e lógica da estrutura, mesmo que isso implicasse maior complexidade na implementação.
- **Comparação case-insensitive de frequências**, para evitar conflitos implícitos e simplificar a verificação de interferência.
- **Separação clara de responsabilidades entre funções**, garantindo que cada função executa apenas uma tarefa (por exemplo, *create_ed()* apenas cria nós, enquanto a verificação é feita antes da inserção).

Estas decisões visaram melhorar a manutenção do sistema, bem como alinhar a estrutura do código com boas práticas de engenharia de software.

Dificuldades Técnicas Sentidas

Durante o desenvolvimento surgiram diversos desafios, sobretudo relacionados com listas ligadas e propagação de interferências:

- **Gestão correta de apontadores em inserção ordenada**, especialmente no tratamento de inserção no início, meio ou fim da lista, exigiu testes cuidadosos e depuração intensiva.
- **Lógica de propagação de efeitos nefastos**, que envolvia criar múltiplos nós em posições calculadas entre duas antenas, revelou-se mais complexa do que inicialmente previsto.
- **Visualização no terminal**, com distinção entre antenas e interferência, exigiu repetidos testes até atingir uma representação clara e compreensível.

Apesar destas dificuldades, todas as funcionalidades fundamentais foram implementadas com sucesso.

Falhas Identificadas e Aprendizagens

A análise posterior ao projeto, em especial após validação com o docente, revelou falhas e aspetos que podem ser otimizados:

- **Repetição de lógica na função `remove_ed()`**, que reimplementa a pesquisa por coordenadas em vez de reutilizar a função `find_ed()`. Esta duplicação contraria o princípio de reutilização de código e poderá ser corrigida com refatoração.
- **Ausência de validação de limites da matriz nas funções de inserção e remoção**, o que permite que coordenadas inválidas (fora do plano) sejam inseridas ou removidas sem verificação. Esta validação deveria ter sido centralizada numa função no módulo de validação.
- **Libertação silenciosa de memória**, sem qualquer verificação durante a execução de `free_ed_list()`, o que dificultou a análise durante testes mais complexos.

Estas falhas evidenciaram a importância de uma abordagem mais defensiva, reutilizável e validada desde o início.

Lições Aprendidas e Melhoria Contínua

Este projeto permitiu consolidar conceitos fundamentais como gestão de memória, manipulação de listas ligadas e organização modular do código. Além disso, destacou a importância de:

- Projetar funções com **responsabilidades únicas**
- **Validar dados de entrada** antes de alterar estruturas
- Reutilizar funções auxiliares e **evitar duplicação de lógica**
- Aplicar ferramentas como o **Doxygen**, tanto para documentação como para apoio à arquitetura do sistema

Propostas de Melhoria Futuras

Durante o desenvolvimento e posterior análise do projeto, surgiram ideias e considerações que, embora não tenham sido implementadas nesta fase, poderão representar melhorias relevantes no futuro:

Utilização de apontador tail

A dada altura chegou a ser implementado um **apontador tail** na estrutura da lista ligada, com o objetivo de acelerar a inserção no fim da lista, evitando a travessia completa da estrutura. Esta ideia, embora **não tenha sido abordada nas aulas**, surgiu de forma espontânea como tentativa de otimização.

Contudo, após testes e análise crítica, concluiu-se que a inserção era feita de forma ordenada, pelo que a inclusão de tail não trazia vantagens reais neste contexto. Além disso, a sua manutenção traria complexidade desnecessária, especialmente durante remoções. Por esse motivo, **decidiu-se remover esta funcionalidade**, mas mantê-la como proposta para cenários com inserção sequencial.

Reestruturação de funções com lógica repetida

A função `remove_ed()` contém lógica de pesquisa duplicada, semelhante à usada em `find_ed()`. Uma futura melhoria poderá passar por criar uma função auxiliar

`find_previous_ed()` ou adaptar `remove_ed()` para reutilizar a função de procura já existente, reduzindo redundância e aumentando a clareza do código.

Generalização com apontadores para função

A função `find_ed()` poderia ser transformada numa função genérica que recebe um **apontador para função de comparação**, permitindo utilizá-la com diferentes critérios (por exemplo, por coordenadas, por frequência, etc.). Esta abordagem aumentaria a reutilização de código e a flexibilidade do sistema.

Considerações Finais da Fase 2

Para a Fase 2, foi desenvolvido um ciclo de testes em modo estático (*Happy Path*), com o objetivo de demonstrar e validar as funcionalidades principais implementadas: leitura da matriz, construção do grafo, ligação automática entre vértices adjacentes com a mesma frequência, percursos em largura (BFS), percursos em profundidade (DFS) e persistência através de ficheiros binários.

Devido à limitação de tempo disponível para esta fase, não foi possível aplicar o mesmo grau de aprofundamento estrutural e de interface que foi alcançado na Fase 1. Por esse motivo, não foi criado um menu gráfico interativo, tendo sido optado por uma abordagem de execução direta no ``main.c`` para efeitos de demonstração.

Além disso, a alínea 3d do enunciado da Fase 2, a análise de interseções entre antenas de diferentes frequências, não foi implementada. Embora tivesse a possibilidade de recorrer a assistência por inteligência artificial para resolver esse ponto, optei conscientemente por limitar o trabalho às funcionalidades que domino, garantindo que todo o código apresentado é compreendido e justificável da minha parte.

Esta escolha permitiu focar na qualidade, clareza e solidez das funcionalidades desenvolvidas, assegurando que a estrutura do grafo, os algoritmos de percurso e a gestão da memória dinâmica foram realizados de forma confiável e alinhada com os princípios ensinados na unidade curricular.

Capítulo 6 – Conclusão

O presente relatório descreveu o desenvolvimento completo do projeto prático de Estruturas de Dados Avançadas, englobando as Fases 1 e 2. O trabalho teve como principal objetivo a implementação e evolução de estruturas dinâmicas em linguagem C, aplicadas à simulação de uma rede de antenas com regras específicas de ligação e comunicação.

Na Fase 1, foram aplicados algoritmos fundamentais de gestão de listas ligadas, com foco na criação, inserção ordenada, pesquisa, remoção e libertação de memória. Esta fase serviu de alicerce para a estruturação da lógica de negócio, associada à deteção de interferências entre antenas de mesma frequência e à inserção de efeitos nefastos. A modularidade e separação de responsabilidades foram priorizadas desde o início, facilitando a manutenção e escalabilidade do sistema.

A Fase 2 representou uma progressão natural e desafiante, introduzindo o conceito de grafo como estrutura central. Nesta fase, foram modelados vértices (antenas), listas de adjacência, e aplicados algoritmos clássicos como a busca em largura (BFS) e profundidade (DFS). Adicionalmente, o projeto passou a permitir persistência de dados através de ficheiros de texto e binários, com mecanismos de validação e marcadores de integridade para reforçar a fiabilidade da leitura.

Ao longo de ambas as fases, foi mantido o compromisso com boas práticas de programação, modularidade, clareza do código e documentação através da ferramenta Doxygen. A experiência revelou-se particularmente valiosa para reforçar o raciocínio algorítmico, a capacidade de abstração, e a atenção à integridade e organização de projetos de média escala em C.

De forma geral, este projeto contribuiu significativamente para o aprofundamento dos conhecimentos sobre estruturas dinâmicas e grafos, promovendo não só competências técnicas, mas também capacidade de decisão e responsabilidade no desenvolvimento de software estruturado.

Referências

- Damas, L. (2002). *Programação em C*. FCA Editora de Informática.
- Horowitz, E., Sahni, S., & Anderson-Freed, S. (2009). *Fundamentos de Estruturas de Dados em C*. Bookman.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Weiss, M. A. (2011). *Data Structures and Algorithm Analysis in C* (3rd ed.). Addison-Wesley.